

HỒ SƠ THIẾT KẾ KIẾN TRÚC

Hệ thống đặt combo Chuyển đi trọn gói

Sinh viên: Đặng Khánh An | Lớp: IT214 | Mã sinh viên: PTIT070

1. Phân tích bài toán

Giao dịch gồm giữ vé máy bay, giữ phòng khách sạn và thanh toán. Mỗi bước thuộc một hệ thống độc lập nên không thể khóa chung dữ liệu hoặc rollback bằng một transaction ACID. Mục tiêu là đạt tính nhất quán cuối: hoặc mọi bước cùng thành công, hoặc những bước đã hoàn tất được bù trừ.

- Combo Orchestrator: nhận yêu cầu, lưu tiến độ Saga và quyết định bù trừ.
- Flight Service: giữ vé; hành động bù trừ là hủy giữ vé.
- Hotel Service: giữ phòng; hành động bù trừ là hủy giữ phòng.
- Payment Service: thu tiền; hành động bù trừ là hoàn tiền.

Đầu vào: customerId, flightCode, hotelCode, amount. Đầu ra: mã combo, trạng thái cuối, mã đặt chỗ và thông báo.

2. Lựa chọn mô hình

Bài chọn Orchestration Saga. Hai đối tác có API, timeout và mã lỗi khác nhau; trình tự giữ vé - giữ phòng - thanh toán cũng rất rõ. Một orchestrator giúp quan sát trạng thái tại một nơi và gọi compensate theo thứ tự ngược. Choreography giảm phụ thuộc trung tâm nhưng sẽ tạo nhiều event và khó xác định bước đang lỗi khi số nhánh tăng.

3. Điểm có thể thất bại

Bước	Rủi ro	Cách xử lý
Giữ vé	Hết chỗ, lỗi mạng	Dừng Saga, hủy combo
Giữ phòng	Hết phòng, timeout	Hủy vé đã giữ
Thanh toán	Bị từ chối, timeout	Hủy phòng rồi hủy vé
Xác nhận cuối	Orchestrator lỗi	Hoàn tiền, hủy phòng, hủy vé
Compensate	Đối tác tạm lỗi	Retry riêng và ghi trạng thái cần xử lý

4. Sơ đồ kiến trúc và luồng

Khách hàng → Combo Orchestrator → Flight API → Hotel API → Payment API

Luồng thành công

1. Tạo combo PENDING. 2. Giữ vé. 3. Giữ phòng. 4. Thanh toán. 5. Chuyển combo sang CONFIRMED.

PENDING → FLIGHT_RESERVED → HOTEL_RESERVED → PAID → CONFIRMED

Luồng bù trừ

Kịch bản	Các bước đã thành công	Bù trừ theo thứ tự ngược
Hotel thất bại	Flight	Cancel Flight → CANCELLED
Hotel timeout	Flight	Hết retry → Cancel Flight → CANCELLED
Payment thất bại	Flight, Hotel	Cancel Hotel → Cancel Flight → CANCELLED
Lỗi sau Payment	Flight, Hotel, Payment	Refund → Cancel Hotel → Cancel Flight

5. Timeout, retry và idempotency

Hotel API có timeout 2 giây. Lỗi mạng hoặc 5xx được retry tối đa 2 lần, cách nhau 500 ms. Lỗi nghiệp vụ HTTP 422 không retry vì thử lại ngay không thay đổi kết quả. Sau khi hết retry, orchestrator chạy compensate.

Các API cancel và refund phải idempotent theo reservationId/paymentId. Gọi lặp lại vẫn trả trạng thái CANCELLED hoặc REFUNDED, không thực hiện nghiệp vụ hai lần. Trong production, Saga state và idempotency key phải lưu bền vững thay vì chỉ dùng RAM.

6. Cấu trúc source demo

- combo-orchestrator (8080): WebClient, SagaContext, timeout, retry và compensate.
- flight-service (8081): reserve/cancel vé.
- hotel-service (8082): reserve/cancel phòng, giả lập lỗi và chậm 5 giây.
- payment-service (8083): charge/refund.

7. Kịch bản nghiệm thu

Scenario	Kết quả mong đợi
SUCCESS	Vé, phòng và thanh toán thành công; combo CONFIRMED.
HOTEL_FAIL	Flight đã giữ được hủy; combo CANCELLED.
PAYMENT_FAIL	Hotel và Flight được hủy; combo CANCELLED.
HOTEL_TIMEOUT	Hotel vượt timeout, hết retry, Flight được hủy.

8. Đánh đổi và hướng production

Orchestration dễ debug và kiểm soát bù trừ, nhưng orchestrator có thể thành nút nghẽn. Cần triển khai nhiều instance, lưu trạng thái Saga bền vững, dùng transactional outbox/message broker, correlationId và distributed tracing. Lệnh compensate thất bại phải được đưa vào hàng đợi retry hoặc DLQ để nhân viên vận hành xử lý.

Không nên retry mù quáng thao tác thanh toán. Mọi request tới đối tác cần idempotency key để tránh trùng lặp hoặc đặt chỗ hai lần khi phản hồi trước bị thất lạc.

9. Kết luận

Saga không tạo tính nguyên tử tức thời như transaction cơ sở dữ liệu, nhưng cung cấp tính nhất quán cuối bằng các giao dịch cục bộ và hành động bù trừ. Với combo có trình tự cố định và nhiều API đối tác, Orchestration là lựa chọn dễ hiểu, dễ kiểm thử và phù hợp nhất cho bản thiết kế này.